# ReviewPyPer React

## Full-Stack Systematic Review Platform

Technical Architecture & API Documentation

| | |
|---|---|
| **Version:** | 2.0 (React Rewrite) |
| **Date:** | March 25, 2026 |
| **Stack:** | React + TypeScript + Tailwind CSS / FastAPI + Python |
| **Original:** | ReviewPyPer (Streamlit) by Calvin Whow |

Comprehensive documentation of every module, function, endpoint, component, and data flow.

# Contents


**1 Executive Summary** **4**
1.1 Architecture Overview . . . 4

**2 System Architecture** **4**
2.1 Directory Structure . . . 4
2.2 Technology Stack . . . 5

**3 Call Graph: Complete System Flow** **6**
3.1 End-to-End Workflow Call Graph . . . 6
3.2 Screening Pipeline Call Graph . . . 7
3.3 LLM Client Call Graph . . . 7

**4 Core Python Modules (Original)** **8**
4.1 LLM Module (`core/llm/`) . . . 8
4.1.1 BaseLLMClient (base_client.py) . . . 8
4.1.2 OpenAIClient (openai_client.py) . . . 8
4.1.3 AnthropicClient (anthropic_client.py) . . . 9
4.1.4 Prompt Templates (prompts.py) . . . 9
4.1.5 CostTracker (cost_tracker.py) . . . 9
4.1.6 Rate Limiting (rate_limit.py) . . . 10
4.2 Storage Module (`core/storage/`) . . . 10
4.2.1 Data Models (models.py) . . . 10
4.2.2 SessionManager (session_manager.py) . . . 11
4.2.3 AuditLogger (audit_logger.py) . . . 12
4.3 Screening Module (`core/screening/`) . . . 12
4.3.1 CriteriaGenerator (criteria_generator.py) . . . 12
4.3.2 TitleAbstractScreener (title_abstract.py) . . . 13
4.3.3 FulltextScreener (fulltext.py) . . . 13
4.3.4 FeedbackReviewer (feedback.py) . . . 13
4.4 Extraction Module (`core/extraction/`) . . . 13
4.4.1 DataExtractor (data_extractor.py) . . . 13
4.4.2 FieldRecommender (field_recommender.py) . . . 14
4.5 Search Strategy Module (`core/search_strategy/`) . . . 14
4.5.1 PICOAnalyzer (pico_analyzer.py) . . . 14
4.5.2 ConceptBuilder (concept_builder.py) . . . 14
4.5.3 PubMedGenerator (pubmed_generator.py) . . . 14
4.5.4 DatabaseTranslator (db_translator.py) . . . 14
4.5.5 SyntaxValidator (syntax_validator.py) . . . 14
4.6 Risk of Bias Module (`core/risk_of_bias/`) . . . 14
4.6.1 RoBAssessor (assessor.py) . . . 14
4.6.2 StudyDesignDetector (study_design_detector.py) . . . 15
4.6.3 TemplateManager (template_manager.py) . . . 15
4.6.4 Built-in RoB Templates . . . 15
4.6.5 Visualization (visualization.py) . . . 15
4.7 PDF Processing Module (`core/pdf/`) . . . 15
4.8 File Parsers Module (`core/file_parsers/`) . . . 16
4.9 Export Module (`core/export/`) . . . 16

**5 FastAPI Backend** **16**
5.1 Application Entry Point (main.py) . . . 16

5.2 State Management (services/state.py) . . . 16
5.3 API Routers . . . 17
5.3.1 Projects Router (routers/projects.py) . . . 17
5.3.2 LLM Router (routers/llm.py) . . . 17
5.3.3 Screening Router (routers/screening.py) . . . 17
5.3.4 Extraction Router (routers/extraction.py) . . . 18
5.3.5 Risk of Bias Router (routers/risk_of_bias.py) . . . 18
5.3.6 Search Strategy Router (routers/search_strategy.py) . . . 18
5.3.7 Export Router (routers/export.py) . . . 19

**6 React Frontend** **19**
6.1 TypeScript Type System (types/index.ts) . . . 19
6.2 API Service Layer (services/api.ts) . . . 19
6.3 React Query Hooks (hooks/useApi.ts) . . . 20
6.4 Layout Components . . . 20
6.4.1 AppLayout (components/layout/AppLayout.tsx) . . . 20
6.4.2 Sidebar (components/layout/Sidebar.tsx) . . . 20
6.5 Reusable UI Components . . . 20
6.6 Page Components (9 Steps) . . . 21
6.6.1 Step 0: SetupReview . . . 21
6.6.2 Step 1: SearchStrategy . . . 21
6.6.3 Step 2: TitleAbstractScreening . . . 22
6.6.4 Step 3: FulltextScreening . . . 22
6.6.5 Step 4: FeedbackReview . . . 22
6.6.6 Step 5: ExtractionSetup . . . 22
6.6.7 Step 6: DataExtraction . . . 22
6.6.8 Step 7: RoBSetup . . . 22
6.6.9 Step 8: RiskOfBias . . . 23
6.7 PRISMA Flow Component (PrismaFlow.tsx) . . . 23
6.8 Routing Configuration (App.tsx) . . . 23

**7 Data Flow Diagrams** **23**
7.1 Complete Data Flow: Reference to Inclusion . . . 24
7.2 LLM Request/Response Flow . . . 24

**8 Database Schema** **25**

**9 Cost Model** **25**
9.1 Per-Operation Cost Estimates . . . 26
9.2 Example: 100-Study Review . . . 26

**10 Configuration Reference** **26**
10.1 Application Settings (config/settings.py) . . . 26
10.2 Environment Variables . . . 27

**11 Deployment** **27**
11.1 Development Setup . . . 27
11.2 Production Build . . . 27

**12 Complete API Endpoint Reference** **28**

**13 Component Dependency Graph** **29**

**14 Summary Statistics** **29**

# 1 Executive Summary

ReviewPyPer React is a full-stack rewrite of the ReviewPyPer systematic review platform. The original application, built with Streamlit, has been decomposed into a **React + TypeScript frontend** and a **FastAPI backend**, with the original Python core modules retained as the business logic layer.

The platform automates the systematic review workflow following **PRISMA 2020 guidelines**, leveraging Large Language Models (OpenAI GPT and Anthropic Claude) for:

- PICO-based screening criteria generation
- Title/abstract and full-text study screening
- Structured data extraction from PDFs
- Risk of Bias assessment using validated tools (RoB 2, ROBINS-I, QUADAS-2, Newcastle-Ottawa, JBI)
- Multi-database search strategy generation and translation

## 1.1 Architecture Overview

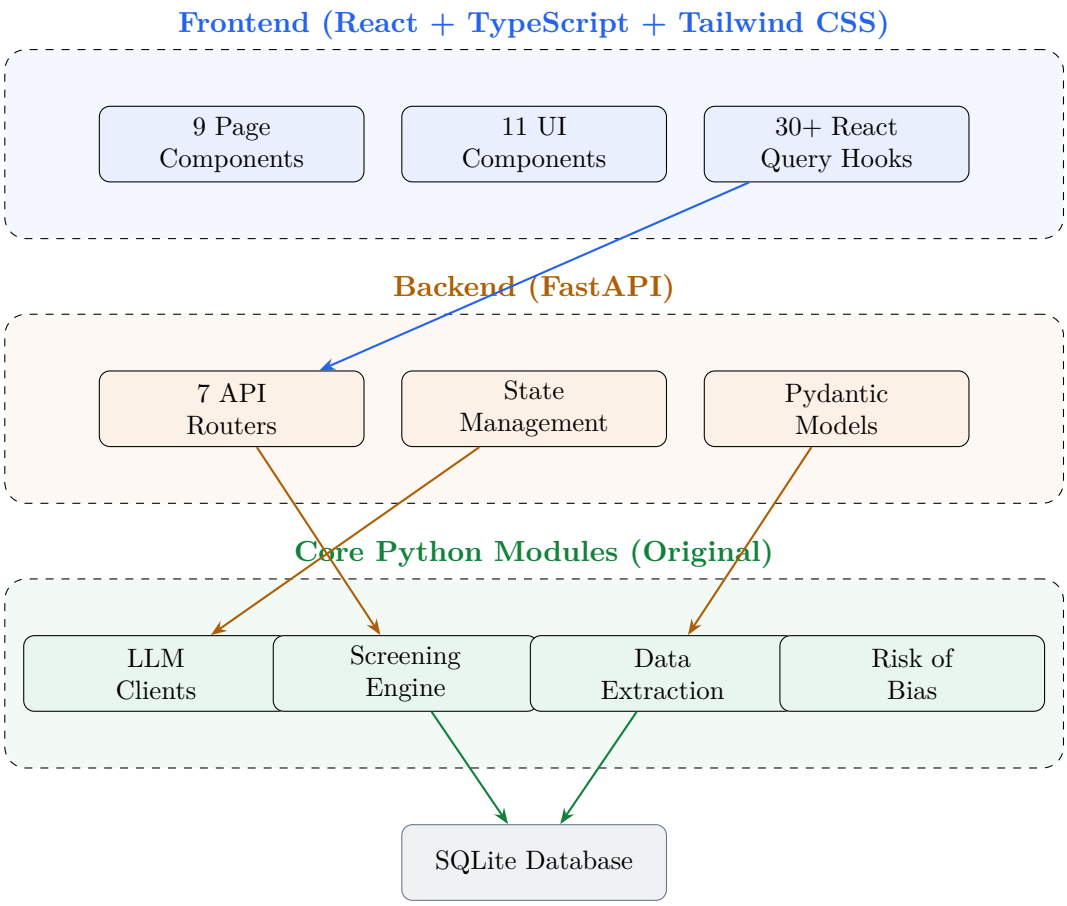


Figure 1: Three-layer architecture: Frontend, Backend API, and Core Python modules.

# 2 System Architecture

## 2.1 Directory Structure

Project Layout

```
reviewpyper-react/
|-- frontend/                    # React + Vite + Tailwind
|   |-- src/
|   |   |-- types/index.ts       # 15 TypeScript interfaces
|   |   |-- services/api.ts      # Axios API client
|   |   |-- hooks/useApi.ts      # 30+ React Query hooks
|   |   |-- components/
|   |   |   |-- layout/          # AppLayout, Sidebar
|   |   |   |-- ui/              # 11 reusable UI components
|   |   |   |-- PrismaFlow.tsx   # PRISMA 2020 diagram
|   |   |-- pages/               # 9 workflow step pages
|   |   |-- App.tsx              # Router + providers
|   |   |-- main.tsx             # Entry point
|   |-- vite.config.ts
|   |-- package.json
|
|-- backend/                     # FastAPI
|   |-- main.py                  # App entry + CORS + routers
|   |-- requirements.txt
|   |-- services/
|   |   |-- state.py             # Global state management
|   |-- routers/                 # 7 API router modules
|       |-- projects.py
|       |-- llm.py
|       |-- screening.py
|       |-- extraction.py
|       |-- risk_of_bias.py
|       |-- search_strategy.py
|       |-- export.py
|
|-- docs/                        # This report
```

## 2.2 Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| Frontend Framework | React 19 + TypeScript | UI rendering |
| Build Tool | Vite 6 | Development server + bundler |
| Styling | Tailwind CSS v4 | Utility-first CSS |
| State/Data | TanStack React Query | Server state management |
| HTTP Client | Axios | API communication |
| Icons | Lucide React | SVG icon library |
| Routing | React Router v7 | Client-side routing |
| Backend Framework | FastAPI | REST API server |
| ASGI Server | Uvicorn | Production server |
| Validation | Pydantic v2 | Request/response models |
| Database | SQLite | Project data persistence |
| LLM Providers | OpenAI SDK, Anthropic SDK | AI capabilities |
| PDF Processing | PyPDF2, Tesseract OCR | Text extraction |
| File Parsing | Custom parsers | CSV, RIS, NBIB, BibTeX, EndNote |

Table 1: Complete technology stack.

# 3 Call Graph: Complete System Flow

## 3.1 End-to-End Workflow Call Graph

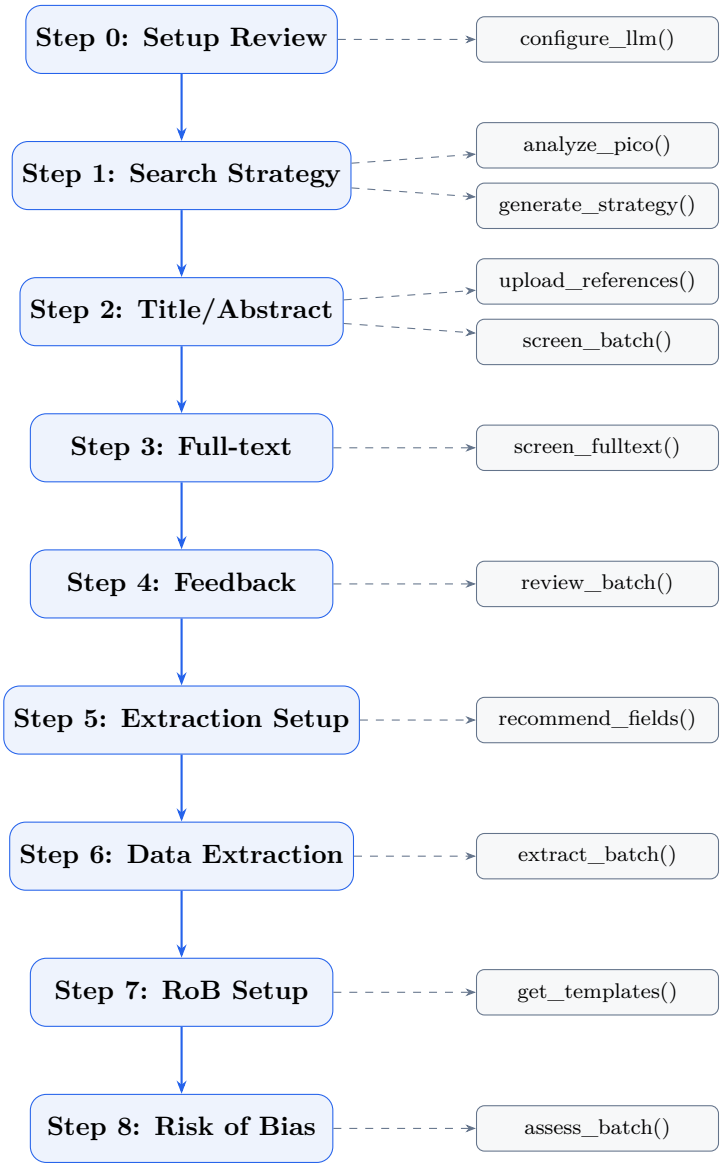


Figure 2: 9-step systematic review workflow with core function calls.

## 3.2 Screening Pipeline Call Graph

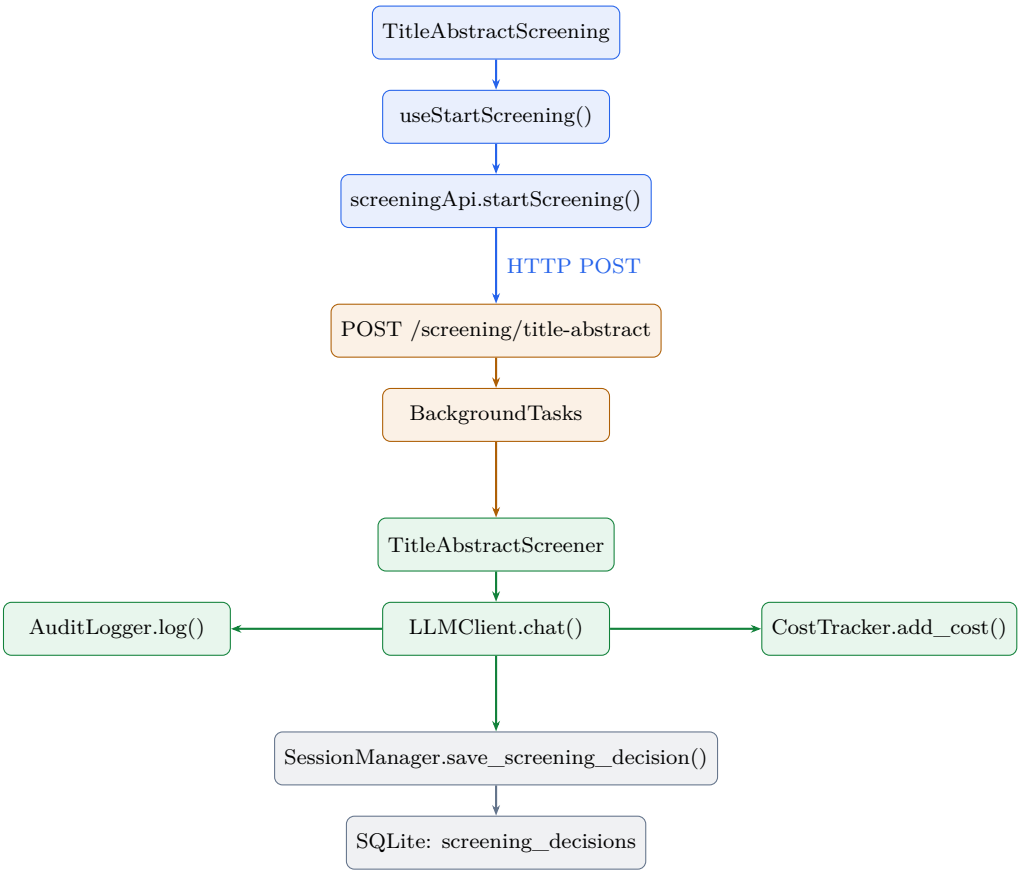


Figure 3: Title/abstract screening call graph from UI click to database persistence.

## 3.3 LLM Client Call Graph

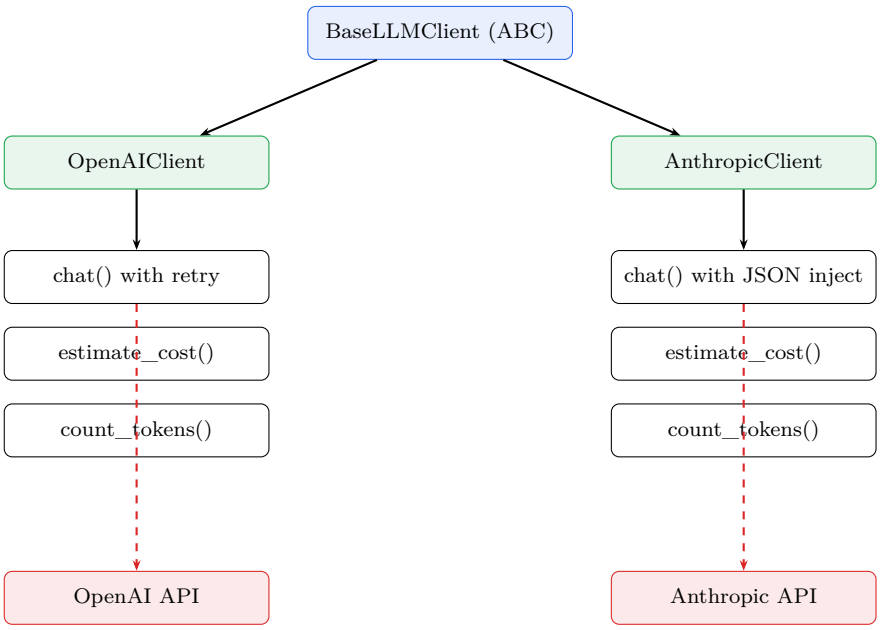


Figure 4: LLM client inheritance and external API calls.

# 4 Core Python Modules (Original)

The core Python modules provide all business logic. The FastAPI backend wraps these modules with HTTP endpoints.

## 4.1 LLM Module (`core/llm/`)

### 4.1.1 BaseLLMClient (base_client.py)

Abstract base class defining the LLM provider interface.

| Method | Signature | Description |
|---|---|---|
| `__init__` | `(api_key, model)` | Initialize with credentials |
| `chat` | `(messages, temperature, max_tokens, json_mode)` → `LLMResponse` | Send chat request (abstract) |
| `estimate_cost` | `(input_tokens, output_tokens)` → `float` | Estimate USD cost (abstract) |
| `count_tokens` | `(text)` → `int` | Count tokens in text (abstract) |
| `provider_name` | property → `str` | Provider identifier (abstract) |
| `supported_models` | property → `list[str]` | Available models (abstract) |

Table 2: BaseLLMClient abstract interface.

**LLMResponse** dataclass: `content: str`, `input_tokens: int`, `output_tokens: int`, `cost: float`.

### 4.1.2 OpenAIClient (openai_client.py)

Implements `BaseLLMClient` for OpenAI GPT models.

- **Default model:** `gpt-4o`
- **Supported models:** GPT-5.2-pro, GPT-5.2, GPT-5, GPT-5-mini, GPT-5-nano, GPT-4.1, GPT-4o, GPT-4o-mini, GPT-4-turbo, GPT-3.5-turbo
- **Retry logic:** Auto-retry on rate limit (429) and server errors (5xx), up to 5 retries with exponential backoff
- **Rate limit parsing:** Extracts wait time from error message ("try again in Xms/Xs")
- **Token counting:** Uses `tiktoken` library with fallback to character-based estimation
- **JSON mode:** Supported via `response_format={"type": "json_object"}`
- **chat_safe():** Graceful error handling that returns a safe fallback on failure

| Model | Input ($/1M) | Output ($/1M) |
|---|---:|---:|
| gpt-5.2-pro | $15.00 | $60.00 |
| gpt-5 | $10.00 | $30.00 |
| gpt-4o | $5.00 | $15.00 |
| gpt-4o-mini | $0.15 | $0.60 |
| gpt-3.5-turbo | $0.50 | $1.50 |

Table 3: OpenAI model pricing (per 1M tokens).

#### 4.1.3 AnthropicClient (anthropic_client.py)

Implements `BaseLLMClient` for Anthropic Claude models.

- **Default model:** `claude-sonnet-4-20250514`
- **Supported models:** Claude Sonnet 4, Claude Opus 4, Claude 3.7 Sonnet, Claude 3.5 Sonnet/Haiku, Claude 3 series
- **Token counting:** Character-based approximation (`CHARS_PER_TOKEN = 4.0`)
- **JSON mode:** Injected via system prompt ("Respond only with valid JSON")
- **System messages:** Extracted from message list and passed as separate `system` parameter

| Model | Input ($/1M) | Output ($/1M) |
|---|---|---|
| claude-opus-4 | $15.00 | $75.00 |
| claude-sonnet-4 | $3.00 | $15.00 |
| claude-3.5-haiku | $0.80 | $4.00 |

Table 4: Anthropic model pricing (per 1M tokens).

#### 4.1.4 Prompt Templates (prompts.py)

294 lines of carefully engineered prompt templates:

| Template | Purpose |
|---|---|
| `CRITERIA_GENERATION_*` | Generate PICO inclusion/exclusion criteria from research question |
| `TITLE_ABSTRACT_SCREENING_*` | Screen study by title+abstract, return JSON decision |
| `FULLTEXT_SCREENING_*` | Detailed full-text screening with per-criterion evaluation |
| `FEEDBACK_REVIEW_*` | Re-review low-confidence exclusions with inclusive prompting |
| `FIELD_RECOMMENDATION_*` | Suggest data extraction fields for research question |
| `DATA_EXTRACTION_*` | Extract structured fields from study text |
| `RISK_OF_BIAS_*` | Domain-based RoB assessment with signaling questions |
| `TRANSLATION_*` | Translate abstracts to English |

Table 5: LLM prompt templates and their purposes.

#### 4.1.5 CostTracker (cost_tracker.py)

Tracks all LLM API costs with budget enforcement.

| Method | Signature | Description |
|---|---|---|
| `__init__` | `(budget_limit=None)` | Initialize tracker |
| `estimate_cost` | `(op_type, count, model)` → `CostEstimate` | Pre-operation estimate |
| `add_cost` | `(op_type, input_tok, output_tok, cost, model)` | Record actual cost |
| `get_summary` | `()` → `dict` | Breakdown by operation |
| `total_cost` | property → `float` | Cumulative USD spent |
| `remaining_budget` | property → `Optional[float]` | Budget remaining |
| `is_paused` | property → `bool` | Budget exceeded flag |

Table 6: CostTracker methods.

**OperationType** enum values: `CRITERIA_GENERATION`, `TITLE_ABSTRACT_SCREENING`, `FULLTEXT_SCREENING`, `FEEDBACK_REVIEW`, `FIELD_RECOMMENDATION`, `DATA_EXTRACTION`, `RISK_OF_BIAS`, `TRANSLATION`, `PICO_ANALYSIS`, `PUBMED_GENERATION`, `DATABASE_TRANSLATION`, `SYNTAX_VALIDATION`, `OTHER`.

**Default token estimates per operation:**

- Title/Abstract screening: 500 input, 100 output tokens
- Full-text screening: 4,000 input, 300 output tokens
- Data extraction: 3,000 input, 500 output tokens
- Risk of Bias: 3,000 input, 400 output tokens
- Criteria generation: 200 input, 500 output tokens

### 4.1.6 Rate Limiting (rate_limit.py)

- **RateLimitConfig**: TPM limits (default 90K), retry settings, backoff parameters
- **TokenBucket**: Thread-safe rolling-window token budget tracker
- **APICallSemaphore**: Singleton limiting concurrent API calls
- **with_retry()**: Retry wrapper with exponential backoff + jitter
- **throttled_api_call()**: Combines TPM throttling with retry logic
- **is_retryable_error()**: Checks for rate limit (429) and server (5xx) errors

## 4.2 Storage Module (`core/storage/`)

### 4.2.1 Data Models (models.py)

Complete Pydantic v2 models for all entities:

| Model | Key Fields |
|---|---|
| `Study` | id, pmid, doi, title, abstract, authors, year, journal, pdf_path, pdf_text, source_database |
| `ScreeningDecision` | study_id, phase, decision, reason, reason_category, confidence, criteria_evaluation, feedback fields |
| `ReviewCriteria` | inclusion (InclusionCriteria), exclusion list, suggested_exclusion_reasons |
| `InclusionCriteria` | population, intervention, comparison, outcome, study_design |
| `ExtractionField` | field_name, description, field_type, category, required, display_order, options |
| `ExtractedValue` | field_name, value, is_not_reported, source_quote, notes, source, confidence |
| `StudyExtraction` | study_id, extractions dict, extraction_quality, timestamps |
| `Project` | id, name, research_question, review_type, criteria, llm settings, budget, prisma_counts |
| `PRISMACounts` | All PRISMA 2020 flow counts |

Table 7: Core data models.

**Enums:**

- `ExclusionCategory`: WRONG_POPULATION, WRONG_INTERVENTION, WRONG_COMPARAT WRONG_OUTCOME, WRONG_STUDY_DESIGN, NOT_ACCESSIBLE, DUPLICATE, OTHER, MEETS_CRITERIA
- `ReviewType`: STANDARD, RAPID, SCOPING
- `ScreeningPhase`: TITLE_ABSTRACT, FULLTEXT
- `FieldType`: TEXT, NUMERIC, CATEGORICAL, DATE, BOOLEAN
- `RoBToolType`: ROB_2, ROBINS_I, NEWCASTLE_OTTAWA_*, QUADAS_2, JBI_*, CUSTOM
- `JudgmentLevel`: LOW, SOME_CONCERNS, MODERATE, SERIOUS, CRITICAL, HIGH, UNCLEAR, NOT_APPLICABLE

**Risk of Bias models:**

- `SignalingQuestion`: question_text, guidance, response_options
- `RoBDomainTemplate`: name, description, signaling_questions, judgment_guidance
- `RoBTemplate`: tool_type, name, description, domains, overall_judgment_algorithm
- `RoBDomainJudgment`: judgment with AI suggestion, human verification, signaling responses
- `StudyRoBAssessment`: Complete assessment with domain judgments, overall judgment, cost
- `RoBProjectSettings`: Enabled tools, dual review mode, auto-detect, batch queue
- `RoBAuditEntry`: Tracks AI vs. human edits for audit trail

**Search Strategy models:**

- `PICOElement`: element_type, label, primary_terms, synonyms, mesh_terms
- `ConceptBlock`: PICO element with boolean operator
- `SearchStrategy`: Multi-database strategies with validation and history
- `ParsedReference`: Unified reference with deduplication fields

### 4.2.2 SessionManager (session_manager.py)

SQLite-backed persistence layer. Creates and manages 13 database tables.

| Method | Returns | Description |
|---|---|---|
| `create_project()` | Project | Initialize new project |
| `load_project()` | Project | Restore project state |
| `save_project()` | None | Persist project changes |
| `list_projects()` | list[dict] | List all projects |
| `delete_project()` | None | Delete project + data |
| `add_studies()` | None | Bulk insert studies |
| `get_studies()` | list[Study] | Query studies with filters |
| `get_study()` | Study | Single study by ID |
| `update_study()` | None | Update study fields |
| `save_screening_decision()` | None | Persist screening result |
| `get_screening_decisions()` | list[ScreeningDecision] | Query with filters |
| `get_low_confidence_exclusions()` | list[ScreeningDecision] | Below threshold |
| `save_extraction()` | None | Store extracted data |
| `get_extractions()` | list[StudyExtraction] | Get all extractions |
| `save_cost_tracker()` | None | Persist cost state |
| `load_cost_tracker()` | CostTracker | Restore cost state |
| `save_search_strategy()` | None | Persist search strategy |
| `load_search_strategy()` | SearchStrategy | Restore strategy |
| `save_rob_assessment()` | None | Store RoB result |
| `get_rob_assessments()` | list | Query assessments |
| `save_rob_settings()` | None | Store RoB config |
| `get_rob_settings()` | RoBProjectSettings | Load config |
| `save_rob_audit()` | None | Log RoB audit entry |
| `get_rob_audit_entries()` | list | Query audit trail |

Table 8: SessionManager methods (complete listing).

**Database tables:** `project`, `studies`, `screening_decisions`, `extractions`, `extraction_fields`, `audit_log`, `cost_tracking`, `rob_assessments`, `rob_judgments`, `rob_templates`, `rob_settings`, `search_strategies`, `parsed_references`.

### 4.2.3 AuditLogger (audit_logger.py)

| Method | Description |
|---|---|
| `log_llm_call()` | Record operation type, prompt, response, tokens, cost, model, study_id, decision |
| `get_entries()` | Query with filters (project, study, operation, date range) |
| `export_audit_trail()` | JSON/CSV export |
| `get_summary()` | Statistics by operation and model |
| `clear()` | Delete entries |

Table 9: AuditLogger methods.

## 4.3 Screening Module (`core/screening/`)

### 4.3.1 CriteriaGenerator (criteria_generator.py)

Generates PICO-based screening criteria from a research question using LLM.

- `generate_criteria(research_question)` → `ReviewCriteria`: Sends research question to LLM, parses structured PICO criteria
- `refine_criteria(criteria, feedback)` → `ReviewCriteria`: Iterative refinement based on user feedback

- `estimate_cost()` → `CostEstimate`: Pre-generation cost estimate

### 4.3.2 TitleAbstractScreener (title_abstract.py)

Core screening engine. Screens studies by title and abstract.

| Method | Signature | Description |
|---|---|---|
| `screen_study` | `(study, criteria)` → `ScreeningDecision` | Screen single study with caching |
| `screen_batch` | `(studies, criteria, callback)` → `list[ScreeningDecision]` | Batch screening with progress |
| `screen_dataframe` | `(df, criteria)` → `list[ScreeningDecision]` | DataFrame-based screening |
| `get_statistics` | `()` → `dict` | Summary statistics |

Table 10: TitleAbstractScreener methods.

**Configuration:**

- `MAX_ABSTRACT_CHARS = 2000` (truncation limit)
- `MAX_TITLE_CHARS = 300`
- Compact prompts: ∼500 input tokens per study
- Cache prevents duplicate API calls for identical studies
- Fallback: returns "included" on LLM error (safe default)

### 4.3.3 FulltextScreener (fulltext.py)

Full-text screening with detailed per-criterion evaluation.

- `MAX_TEXT_CHARS = 50,000` – truncation keeps beginning + end for context
- `screen_study(study, criteria)` → `ScreeningDecision`: Evaluates each criterion individually
- `screen_batch(studies, criteria, callback)`: Batch with budget control
- Returns `criteria_evaluation`: `{criterion: {met: bool, notes: str}}`
- Tracks `not_accessible` studies (PDF processing failures)

### 4.3.4 FeedbackReviewer (feedback.py)

Re-reviews low-confidence exclusions with inclusive prompting.

- `CONFIDENCE_THRESHOLD = 0.8` (configurable)
- `get_studies_for_review()`: Filter exclusions below threshold
- `review_decision()`: Reconsider with inclusive framing
- `review_batch()`: Batch reconsideration
- `apply_user_overrides()`: Apply manual include/exclude overrides

## 4.4 Extraction Module (core/extraction/)

### 4.4.1 DataExtractor (data_extractor.py)

Extracts structured data from study full-texts.

- `MAX_TEXT_CHARS = 50,000`
- `extract_from_study(study, fields)` → `StudyExtraction`: Single study extraction

- `extract_batch(studies, fields, callback)`: Multiple studies with progress
- Handles "NR" (not reported) fields
- Returns completeness scores per study

#### 4.4.2 FieldRecommender (field_recommender.py)

- `recommend_fields(research_question, study_types)` → `list[ExtractionField]`: LLM-recommended extraction fields based on research question and study types
- Returns structured `ExtractionField` objects with name, description, type, category

### 4.5 Search Strategy Module (`core/search_strategy/`)

#### 4.5.1 PICOAnalyzer (pico_analyzer.py)

- `analyze(research_question)` → `list[PICOElement]`: Extract Population, Intervention, Comparator, Outcome elements with synonyms and MeSH terms

#### 4.5.2 ConceptBuilder (concept_builder.py)

- `build_blocks(pico_elements)` → `list[ConceptBlock]`: Organize PICO elements into searchable concept blocks with OR operators

#### 4.5.3 PubMedGenerator (pubmed_generator.py)

- `generate_pubmed_syntax(concept_blocks)` → `str`: Generate PubMed Boolean search string
- Format: `(P1 OR P2)[tiab] AND (I1 OR I2)[tiab] AND ...`
- Adds MeSH terms with `[mh]` modifiers
- `validate_syntax(query)` → `bool`: Check PubMed syntax validity

#### 4.5.4 DatabaseTranslator (db_translator.py)

Translates PubMed queries to other database syntaxes:

- `translate_pubmed_to_scopus()`: SCOPUS field codes (TITLE-ABS-KEY)
- `translate_pubmed_to_wos()`: Web of Science syntax (TS=)
- `translate_pubmed_to_embase()`: EMBASE with Emtree terms
- `translate_pubmed_to_cochrane()`: Cochrane Library syntax
- `translate_pubmed_to_ovid()`: OVID MEDLINE syntax

#### 4.5.5 SyntaxValidator (syntax_validator.py)

- `validate(strategy, database)` → `ValidationResult`: Database-specific syntax validation
- Returns errors (blocking) and warnings (non-blocking)
- Checks bracket matching, field code validity, operator usage

### 4.6 Risk of Bias Module (`core/risk_of_bias/`)

#### 4.6.1 RoBAssessor (assessor.py)

AI-assisted risk of bias assessment engine.

| Method | Description |
|---|---|
| `assess_study()` | Single study assessment with auto design detection |
| `assess_batch()` | Batch processing with progress callback |

Table 11: RoBAssessor methods.

- Auto-detects study design to select appropriate tool
- Evaluates each domain with signaling questions
- Returns `StudyRoBAssessment` with domain judgments
- Supports dual-review workflow (AI + human)
- Tracks cost per assessment

### 4.6.2 StudyDesignDetector (study_design_detector.py)

- `detect_design(study_text)` → `(design_type, confidence)`: Automatically identifies RCT, cohort, case-control, cross-sectional, diagnostic accuracy, qualitative

### 4.6.3 TemplateManager (template_manager.py)

- `get_template(tool_type)` → `RoBTemplate`: Load built-in template
- `customize_template(template, changes)` → `RoBTemplate`: Project-specific customization
- `save_template(template)`: Persist customization to SQLite

### 4.6.4 Built-in RoB Templates

| Tool | Domains | Applicable Study Designs |
|---|---|---|
| RoB 2 | 5 | Randomized controlled trials |
| ROBINS-I | 7 | Non-randomized studies of interventions |
| QUADAS-2 | 4 | Diagnostic accuracy studies |
| Newcastle-Ottawa | 3 (8 items) | Cohort and case-control studies |
| JBI | Variable | Multiple designs (qualitative, prevalence, etc.) |

Table 12: Available Risk of Bias assessment tools.

### 4.6.5 Visualization (visualization.py)

- Traffic light plots: colored dots (green/yellow/red) per domain per study
- Summary tables: aggregated judgment distribution
- Forest plot data preparation for meta-analysis

## 4.7 PDF Processing Module (`core/pdf/`)

`PDFProcessor` extracts text from PDF files:

- `extract_text(pdf_path)` → `str`: Primary method
- **Method 1:** Direct extraction via PyPDF2
- **Method 2:** OCR fallback via pdf2image + Tesseract (for scanned PDFs)
- Automatic method selection based on word count threshold
- `process_directory(dir_path)` → `dict[str, str]`: Batch processing

### 4.8 File Parsers Module (core/file_parsers/)

| Parser | Format | Key Features |
|---|---|---|
| CSVReferenceParser | CSV | Auto-detection of column mapping, handles Excel exports |
| RISParser | RIS | PubMed/Scopus reference manager format |
| NBIBParser | NBIB | PubMed MEDLINE format with MeSH extraction |
| BibTeXParser | BibTeX | LaTeX bibliography format |
| EndNoteXMLParser | XML | EndNote export format |
| Deduplicator | – | DOI exact match, title fuzzy match, author-year match |

Table 13: Reference file parsers.

### 4.9 Export Module (core/export/)

DocxGenerator creates Word documents:

- generate_prisma_report(): PRISMA 2020 flow diagram in DOCX
- generate_evidence_table(): Formatted data extraction table
- generate_risk_of_bias_report(): RoB summary with formatting
- Supports custom headers, footers, and styles

## 5 FastAPI Backend

### 5.1 Application Entry Point (main.py)

- CORS middleware allows: localhost:5173, localhost:3000
- Adds systematic_review_app to sys.path for core module imports
- Health check: GET /api/health → {"status": "ok"}
- Includes 7 API routers

### 5.2 State Management (services/state.py)

Singleton-pattern module managing global application state:

| Function | Signature | Description |
|---|---|---|
| configure_llm | (provider, api_key, model) → BaseLLMClient | Create & store LLM client |
| get_llm | () → Optional[BaseLLMClient] | Get current client |
| require_llm | () → BaseLLMClient | Get client or raise error |
| get_llm_config | () → dict | Current LLM config |
| get_cost_tracker | () → CostTracker | Global cost tracker |
| reset_cost_tracker | (budget_limit) → CostTracker | Reset with new budget |
| get_session_manager | (project_id, path) → SessionManager | Per-project manager |
| remove_session_manager | (project_id) | Remove from cache |

Table 14: State management functions.

## 5.3 API Routers

### 5.3.1 Projects Router (`routers/projects.py`)

| Method | Endpoint | Description | Status |
|---|---|---|---|
| POST | `/api/projects` | Create project | 201 |
| GET | `/api/projects` | List all projects | 200 |
| GET | `/api/projects/{id}` | Get project details | 200 |
| DELETE | `/api/projects/{id}` | Delete project | 204 |

Table 15: Projects API endpoints.

**Request model – ProjectCreate:**

- `name: str` (1–200 chars)
- `research_question: str` (required)
- `review_type: str` – "standard" | "rapid" | "scoping"
- `storage_path: Optional[str]`

**Response model – ProjectDetail:** Extends ProjectSummary with `storage_path`, `llm_provider`, `llm_model`, `budget_limit`, `has_criteria`, `study_count`.

### 5.3.2 LLM Router (`routers/llm.py`)

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/llm/configure` | Set provider, API key, model |
| GET | `/api/llm/models` | List models with pricing |
| POST | `/api/llm/test` | Test LLM connection |
| GET | `/api/llm/costs` | Get cost tracking summary |

Table 16: LLM API endpoints.

**Model pricing data:** Contains complete pricing for 13 OpenAI models and 12 Anthropic models (see Section 4.1.2 and 4.1.3 for pricing tables).

### 5.3.3 Screening Router (`routers/screening.py`)

The most complex router with 9 endpoints:

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/projects/{id}/criteria` | Generate or save criteria |
| GET | `/api/projects/{id}/criteria` | Get stored criteria |
| POST | `/api/projects/{id}/references/upload` | Upload reference files |
| GET | `/api/projects/{id}/references` | List references (paginated) |
| POST | `/api/projects/{id}/screening/title-abstract` | Start T/A screening |
| GET | `/api/projects/{id}/screening/title-abstract/status` | Get screening progress |
| POST | `/api/projects/{id}/screening/fulltext` | Start FT screening |
| GET | `/api/projects/{id}/screening/results` | Get results with filters |
| POST | `/api/projects/{id}/screening/feedback` | Submit user overrides |

Table 17: Screening API endpoints.

**Background task execution:** Title/abstract and full-text screening run as FastAPI `BackgroundTasks`. Clients poll `.../status` endpoint for progress.

**File format support:** CSV, RIS, NBIB, BibTeX (.bib), EndNote XML (.xml). Auto-detected by file extension.

**Screening statistics:** inclusion rate, average confidence, low confidence count, exclusion by category breakdown.

### 5.3.4 Extraction Router (`routers/extraction.py`)

| Method | Endpoint | Description |
|---|---|---|
| POST | `.../fields/recommend` | AI-recommended fields |
| POST | `.../fields` | Save field configuration |
| GET | `.../fields` | Get configured fields |
| POST | `.../run` | Run extraction (background) |
| GET | `.../results` | Get extraction results |

Table 18: Extraction API endpoints (prefix: `/api/projects/{id}/extraction`).

**Default fields (fallback):** Author, Year, Study Design, Sample Size, Population, Intervention, Comparator, Primary Outcome, Key Findings.

### 5.3.5 Risk of Bias Router (`routers/risk_of_bias.py`)

| Method | Endpoint | Description |
|---|---|---|
| GET | `/api/rob/templates` | List RoB tools |
| POST | `/api/projects/{id}/rob/assess` | Run RoB assessment |
| GET | `/api/projects/{id}/rob/results` | Get assessment results |

Table 19: Risk of Bias API endpoints.

**Assessment statistics:** Distribution by overall judgment, flagged uncertain count, human verified count.

### 5.3.6 Search Strategy Router (`routers/search_strategy.py`)

| Method | Endpoint | Description |
|---|---|---|
| POST | `.../search/pico` | Analyze PICO elements |
| POST | `.../search/generate` | Generate PubMed query |
| POST | `.../search/translate` | Translate to other DBs |
| POST | `.../search/validate` | Validate syntax |

Table 20: Search Strategy API endpoints (prefix: `/api/projects/{id}`).

#### 5.3.7 Export Router (routers/export.py)

| Method | Endpoint | Description |
|---|---|---|
| GET | .../export/csv | Download CSV results |
| GET | .../export/prisma | Get PRISMA flow counts |

Table 21: Export API endpoints (prefix: /api/projects/{id}).

**PRISMA counts model:** records_identified_databases, records_removed_duplicates, records_screened, records_excluded_screening, reports_sought, reports_not_retrieved, reports_assessed, reports_excluded, studies_included, exclusion_reasons.

# 6 React Frontend

## 6.1 TypeScript Type System (types/index.ts)

15 interfaces defining the complete data model for the frontend:

Key Type Definitions

```typescript
interface Project {
  id: string; name: string;
  research_question: string;
  review_type: 'standard' | 'rapid' | 'scoping';
  created_at: string; updated_at: string;
  status: string; llm_config?: LLMConfig;
}

interface ScreeningDecision {
  study_id: string;
  decision: 'include' | 'exclude' | 'pending';
  confidence: number; reason: string;
  category: string;
  criteria_evaluations?: CriterionEvaluation[];
}

interface ExtractionField {
  id?: string; name: string; description: string;
  field_type: 'text'|'numeric'|'categorical'|'date'|'boolean';
  category: 'study_characteristics'|'population'|
            'intervention'|'outcomes'|'other';
  required: boolean; options?: string[];
}

interface RoBAssessment {
  study_id: string; study_title: string;
  tool: string;
  overall_judgment: 'low' | 'some_concerns' | 'high';
  domain_assessments: DomainAssessment[];
}
```

## 6.2 API Service Layer (services/api.ts)

Axios-based client with 7 API modules containing 30+ methods:

| Module | Methods |
|---|---|
| `projectsApi` | list, get, create, update, delete |
| `llmApi` | configure, test, listModels |
| `screeningApi` | setCriteria, getCriteria, uploadReferences, startScreening, getProgress, getResults, submitFeedback, uploadFulltext, getCostEstimate |
| `extractionApi` | getFields, setFields, recommendFields, run, getResults, updateResult |
| `robApi` | getTemplates, setTemplate, assess, getResults |
| `searchApi` | analyzePICO, generateQuery, translateQuery, validateQuery |
| `exportApi` | csv, prisma |
| `costApi` | getSummary |

Table 22: API service modules and their methods.

## 6.3 React Query Hooks (hooks/useApi.ts)

30+ hooks wrapping every API function with proper caching, invalidation, and polling:

- **Queries** (read operations): Auto-cached with 30s stale time, retry once
- **Mutations** (write operations): Invalidate related queries on success
- **Polling**: `useScreeningProgress` auto-refetches every 2s until completed
- **Cost tracking**: `useCostSummary` refetches every 30s

**Key hooks:**

- `useProjects()`, `useProject(id)` – Project data
- `useCreateProject()`, `useDeleteProject()` – Project mutations
- `useConfigureLLM()`, `useTestLLM()` – LLM setup
- `useStartScreening()`, `useScreeningProgress(id, taskId)` – Screening
- `useRunExtraction()`, `useExtractionResults(id)` – Extraction
- `useRunRoB()`, `useRoBResults(id)` – Risk of Bias
- `useAnalyzePICO()`, `useGenerateQuery()` – Search Strategy

## 6.4 Layout Components

### 6.4.1 AppLayout (components/layout/AppLayout.tsx)

- Props: `projectId?: string`
- Uses `useProject()` hook to load project data
- Renders: Sidebar (left) + Header bar (top) + `<Outlet />` (content)
- Header shows: project name, review type badge, creation date
- Responsive: sidebar fixed on desktop, overlay on mobile

### 6.4.2 Sidebar (components/layout/Sidebar.tsx)

- 9-step navigation matching the systematic review workflow
- Step numbers (0–8) with active highlighting
- Uses `useCostSummary()` for cost display at bottom
- Cost breakdown: total, screening, extraction, RoB
- Mobile: hamburger toggle with overlay

## 6.5 Reusable UI Components

11 professionally designed, accessible UI components:

| Component | Props | Description |
|---|---|---|
| `Button` | variant, size, loading, icon | 4 variants (primary/secondary/danger/ghost), 3 sizes, spinner |
| `Card` | children, className | Compound: Card, CardHeader, CardTitle, CardBody, CardFooter |
| `Badge` | variant, dot, children | 6 colors + DecisionBadge + RiskBadge wrappers |
| `Input` | label, error, hint, icon | Input, Textarea, Select subcomponents |
| `Modal` | open, onClose, title, size | Backdrop, ESC key, 4 size variants |
| `ProgressBar` | value, label, color, size | Animated bar with percentage |
| `FileUpload` | accept, multiple, onFiles | Drag-and-drop zone with file list |
| `DataTable` | columns, data, searchable | Sortable, filterable, paginated table |
| `Tabs` | tabs, activeKey, onChange | Tab navigation with count badges |
| `Alert` | variant, title, dismissible | 4 variants with icons |
| `Spinner` | size, label | Loading spinner + FullPageSpinner |

Table 23: Reusable UI component library.

## 6.6 Page Components (9 Steps)

### 6.6.1 Step 0: SetupReview

**Sections:**

1. **Project Configuration**: Name input, research question textarea, review type dropdown (Standard/Rapid/Scoping)
2. **LLM Configuration**: Provider toggle (OpenAI/Anthropic), API key (password field), model dropdown (dynamically loaded), budget limit
3. **Connection Test**: Test button with success/failure feedback
4. **Navigation**: "Continue to Search Strategy" button

**Hooks:** `useCreateProject`, `useUpdateProject`, `useProject`, `useConfigureLLM`, `useTestLLM`, `useLLMModels`

**State:** name, question, reviewType, provider, apiKey, model, budgetLimit, projectId, testResult

### 6.6.2 Step 1: SearchStrategy

**Sections:**

1. **PICO Analysis**: Button to analyze research question → displays P/I/C/O in editable cards
2. **Concept Builder**: Editable blocks with synonyms, add/remove terms
3. **Query Generation**: Generate PubMed query, syntax-highlighted display
4. **Database Tabs**: PubMed, SCOPUS, WOS, Cochrane, EMBASE with translated queries
5. **Actions**: Copy to clipboard, validate syntax

**Hooks:** `useAnalyzePICO`, `useGenerateQuery`, `useTranslateQuery`, `useValidateQuery`

#### 6.6.3 Step 2: TitleAbstractScreening

**Sections:**

1. **Import**: Drag-and-drop file upload (CSV/RIS/NBIB), import summary
2. **Controls**: Batch size selector, cost estimate, "Start Screening" button
3. **Progress**: Real-time progress bar during screening
4. **Results**: Expandable cards with title, abstract, decision badge, confidence bar, reason, category
5. **Filters**: All/Included/Excluded, search, confidence range

**Hooks:** `useUploadReferences`, `useStartScreening`, `useScreeningProgress` (2s polling), `useScreeningResults`, `useCostEstimate`

#### 6.6.4 Step 3: FulltextScreening

**Sections:**

1. **PDF Upload**: Multi-file upload zone (100MB limit)
2. **Controls**: Run screening button with progress
3. **Results**: Decision cards with expandable criteria evaluation (Met/Not Met per criterion)

#### 6.6.5 Step 4: FeedbackReview

**Sections:**

1. **Low-Confidence List**: Studies excluded with <70% confidence
2. **Per-Item Controls**: Confirm exclusion, override to Include, Reset
3. **Batch Actions**: Confirm all exclusions
4. **Submit**: Persist feedback overrides

#### 6.6.6 Step 5: ExtractionSetup

**Sections:**

1. **AI Recommendations**: Button to get LLM-suggested fields
2. **Field Editor**: Inline rows with name, description, type dropdown, category dropdown, required checkbox, remove button
3. **Add Field**: New empty row button
4. **Save**: Persist field configuration

#### 6.6.7 Step 6: DataExtraction

**Sections:**

1. **Run**: Extract data from included studies
2. **Results**: Expandable cards per study showing extracted field values
3. **Inline Edit**: Click to edit values with save/cancel
4. **Export**: Download as CSV

#### 6.6.8 Step 7: RoBSetup

**Sections:**

1. **Tool Selection**: Radio-style cards for RoB 2, ROBINS-I, QUADAS-2, Newcastle-Ottawa, JBI
2. **Tool Info**: Description, applicable study designs, number of domains

3. **Domain Preview**: Expandable showing domain names and signaling questions

#### 6.6.9 Step 8: RiskOfBias

**Sections:**

1. **Run Assessment**: Button with progress
2. **Traffic Light Table**: Studies as rows, domains as columns, colored circles (green=Low, yellow=Some Concerns, red=High)
3. **Detailed Results**: Expandable per-study cards with domain-by-domain judgment, rationale, signaling question responses

### 6.7 PRISMA Flow Component (PrismaFlow.tsx)

Visualizes the PRISMA 2020 systematic review flow diagram:

- **Identification**: Total records identified, duplicates removed
- **Screening**: Records screened, excluded (with reason breakdown)
- **Eligibility**: Full-text assessed, excluded (with reason breakdown)
- **Included**: Final study count
- Connected boxes with directional arrows

### 6.8 Routing Configuration (App.tsx)

Route Definitions

```
Routes:
  /               -> SetupReview
  /setup          -> SetupReview
  /search         -> SearchStrategy
  /screening      -> TitleAbstractScreening
  /fulltext       -> FulltextScreening
  /feedback       -> FeedbackReview
  /extraction-setup -> ExtractionSetup
  /extraction     -> DataExtraction
  /rob-setup      -> RoBSetup
  /rob            -> RiskOfBias
```

All routes wrapped in `AppLayout` with sidebar navigation.

## 7 Data Flow Diagrams

## 7.1 Complete Data Flow: Reference to Inclusion

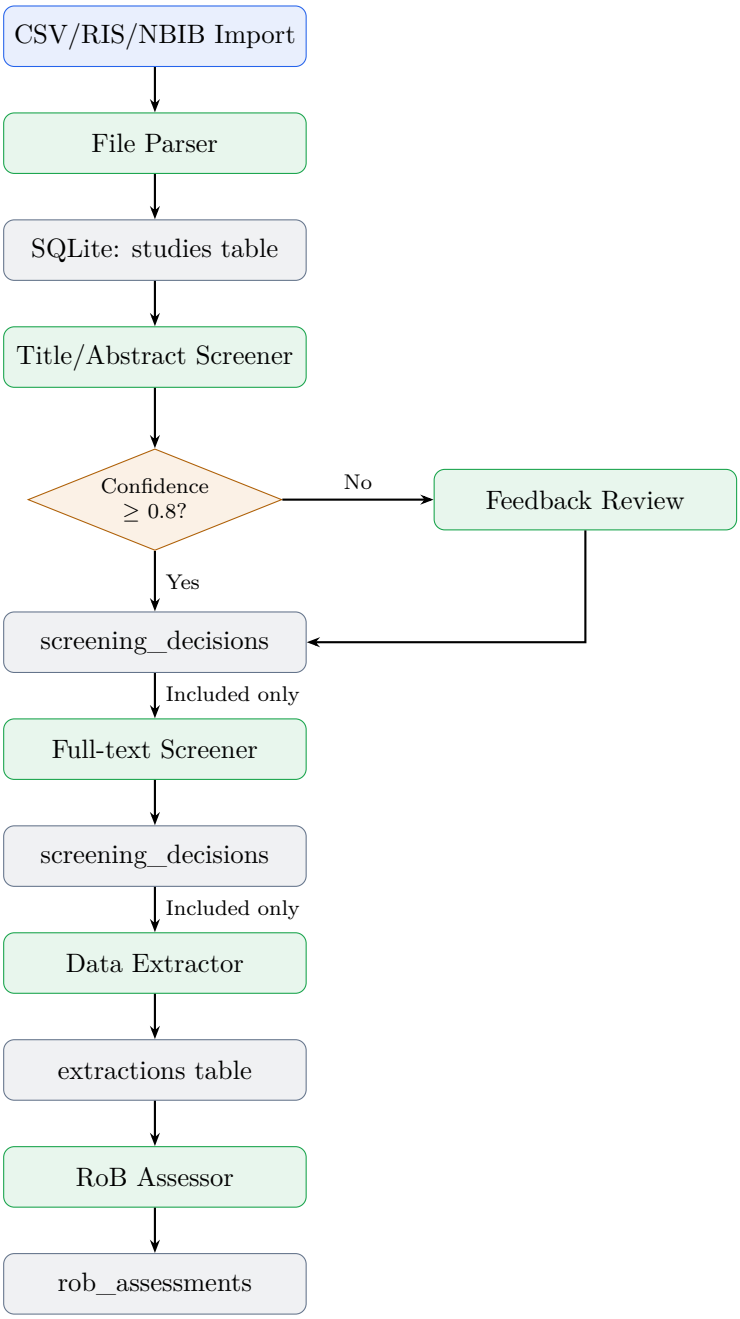


Figure 5: Complete data flow from reference import to final assessment.

## 7.2 LLM Request/Response Flow

For every LLM-powered operation, the following flow occurs:

1. **Cost estimation**: `CostTracker.estimate_cost()` checks budget
2. **Prompt construction**: Operation-specific template filled with data
3. **Rate limiting**: `TokenBucket.wait_for_capacity()` (OpenAI only)
4. **API call**: `LLMClient.chat(messages, json_mode=True)`
5. **Retry**: On 429/5xx errors, exponential backoff up to 5 retries
6. **Response parsing**: JSON extraction from LLM response
7. **Cost recording**: `CostTracker.add_cost()` with actual token counts

8. **Audit logging**: `AuditLogger.log_llm_call()` with full prompt/response
9. **Persistence**: `SessionManager.save_*()` to SQLite

# 8 Database Schema

| Table | Key Columns |
|---|---|
| `project` | id, name, research_question, review_type, criteria (JSON), llm_provider, llm_model, budget_limit, current_phase, timestamps |
| `studies` | id, pmid, doi, title, abstract, authors, year, journal, pdf_path, pdf_text, source_database, created_at |
| `screening_decisions` | id, study_id, phase, decision, reason, reason_category, confidence, criteria_evaluation (JSON), feedback fields |
| `extractions` | id, study_id, field_name, value, is_not_reported, source_quote, notes, source, confidence |
| `extraction_fields` | id, field_name, description, field_type, category, required, display_order, options (JSON) |
| `audit_log` | id, operation_type, prompt, response, input_tokens, output_tokens, cost, model, study_id, decision, timestamp |
| `cost_tracking` | id, operation_type, input_tokens, output_tokens, cost, model, timestamp |
| `rob_assessments` | id, study_id, tool_type, overall_judgment, overall_rationale, ai_cost, assessment_status, assessor_info |
| `rob_judgments` | id, assessment_id, domain_name, judgment_level, rationale, signaling_responses (JSON), ai_confidence |
| `rob_templates` | id, tool_type, name, description, domains (JSON), version |
| `rob_settings` | id, project_id, enabled_tools (JSON), dual_review, auto_detect, batch_queue (JSON) |
| `search_strategies` | id, project_id, pico_analysis (JSON), concept_blocks (JSON), database strategies (JSON) |
| `parsed_references` | id, project_id, source_file, title, abstract, authors, year, journal, doi, pmid, dedup_status |

Table 24: Complete SQLite database schema (13 tables).

# 9 Cost Model

### 9.1 Per-Operation Cost Estimates

| Operation | Input Tokens | Output Tokens | Cost (GPT-4o-mini) |
|---|---|---|---|
| Title/Abstract Screening | 500 | 100 | $0.0001 |
| Full-text Screening | 4,000 | 300 | $0.0008 |
| Data Extraction | 3,000 | 500 | $0.0008 |
| Risk of Bias | 3,000 | 400 | $0.0007 |
| Criteria Generation | 200 | 500 | $0.0003 |
| PICO Analysis | 200 | 400 | $0.0003 |
| PubMed Generation | 500 | 600 | $0.0004 |

Table 25: Per-study cost estimates at GPT-4o-mini pricing.

### 9.2 Example: 100-Study Review

| Phase | Studies | Estimated Cost |
|---|---|---|
| Title/Abstract Screening | 100 | $0.01 |
| Full-text Screening (50 included) | 50 | $0.04 |
| Data Extraction | 50 | $0.04 |
| Risk of Bias Assessment | 50 | $0.04 |
| **Total (GPT-4o-mini)** | | **$0.13** |
| **Total (GPT-4o)** | | **$4.25** |
| **Total (Claude Sonnet 4)** | | **$2.55** |

Table 26: Total cost scaling for a 100-study systematic review.

## 10 Configuration Reference

### 10.1 Application Settings (config/settings.py)

Default Configuration

```python
@dataclass
class LLMSettings:
    openai_api_key: Optional[str] = None
    openai_default_model: str = "gpt-4o"
    anthropic_api_key: Optional[str] = None
    anthropic_default_model: str = "claude-sonnet-4-20250514"
    default_temperature: float = 0.3
    default_max_tokens: int = 1000

@dataclass
class ScreeningSettings:
    low_confidence_threshold: float = 0.8
    auto_include_threshold: float = 0.95
    batch_size: int = 50
    max_retries: int = 3

@dataclass
class ExtractionSettings:
    max_text_chars: int = 50000
    ocr_enabled: bool = True
```

```python
    ocr_dpi: int = 200

@dataclass
class StorageSettings:
    default_storage_path: str = "~/systematic_reviews"
    database_name: str = "project.db"
```

## 10.2 Environment Variables

| Variable | Description | Default |
|---|---|---|
| `OPENAI_API_KEY` | OpenAI API key | None |
| `ANTHROPIC_API_KEY` | Anthropic API key | None |
| `DEBUG` | Debug mode | false |
| `STORAGE_PATH` | Custom storage directory | `~/systematic_reviews` |
| `OPENAI_TPM_LIMIT` | Tokens per minute | 90000 |
| `SCREENING_MAX_ABSTRACT_CHARS` | Abstract truncation | 2000 |
| `SCREENING_MAX_TITLE_CHARS` | Title truncation | 300 |

Table 27: Environment variable reference.

# 11 Deployment

## 11.1 Development Setup

**Backend:**

Backend Setup

```bash
cd reviewpyper-react/backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# API docs: http://localhost:8000/docs
```

**Frontend:**

Frontend Setup

```bash
cd reviewpyper-react/frontend
npm install
npm run dev
# Opens: http://localhost:5173
```

## 11.2 Production Build

Production Build

```bash
# Frontend
cd frontend && npm run build
# Output: dist/ (38.8 KB CSS, 386 KB JS, 116 KB gzipped)

# Backend
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

# 12 Complete API Endpoint Reference

| Method | Endpoint | Request Body | Response |
|---|---|---|---|
| GET | /api/health | – | {status: ok} |
| **Projects** | | | |
| POST | /api/projects | ProjectCreate | ProjectDetail |
| GET | /api/projects | – | ProjectSummary[] |
| GET | /api/projects/{id} | – | ProjectDetail |
| DELETE | /api/projects/{id} | – | 204 |
| **LLM** | | | |
| POST | /api/llm/configure | LLMConfigureRequest | LLMConfigureResponse |
| GET | /api/llm/models | – | ModelInfo[] |
| POST | /api/llm/test | LLMTestRequest | LLMTestResponse |
| GET | /api/llm/costs | – | CostSummary |
| **Screening** | | | |
| POST | .../criteria | CriteriaRequest | CriteriaOut |
| GET | .../criteria | – | CriteriaOut |
| POST | .../references/upload | multipart/form-data | {count} |
| GET | .../references | ?page&page_size | ReferenceListResponse |
| POST | .../screening/title-abstract | – | ScreeningStatusResponse |
| GET | .../screening/ta/status | ?task_id | ScreeningStatusResponse |
| POST | .../screening/fulltext | FulltextScreeningRequest | ScreeningStatusResponse |
| GET | .../screening/results | ?phase&decision&min_con | ScreeningResultsResponse |
| POST | .../screening/feedback | FeedbackRequest | {updated} |
| **Extraction** | | | |
| POST | .../extraction/fields/recommend | FieldRecommendRequest | ExtractionFieldOut[] |
| POST | .../extraction/fields | FieldSaveRequest | ExtractionFieldOut[] |
| GET | .../extraction/fields | – | ExtractionFieldOut[] |
| POST | .../extraction/run | – | ExtractionStatusResponse |
| GET | .../extraction/results | – | ExtractionResultsResponse |
| **Risk of Bias** | | | |
| GET | /api/rob/templates | – | RoBTemplateInfo[] |
| POST | .../rob/assess | RoBAssessRequest | RoBStatusResponse |
| GET | .../rob/results | – | RoBResultsResponse |
| **Search Strategy** | | | |
| POST | .../search/pico | PICORequest | PICOResponse |
| POST | .../search/generate | GenerateRequest | GenerateResponse |
| POST | .../search/translate | TranslateRequest | TranslateResponse |
| POST | .../search/validate | ValidateRequest | ValidateResponse |
| **Export** | | | |
| GET | .../export/csv | ?phase | text/csv |
| GET | .../export/prisma | – | PRISMACountsOut |

# 13 Component Dependency Graph

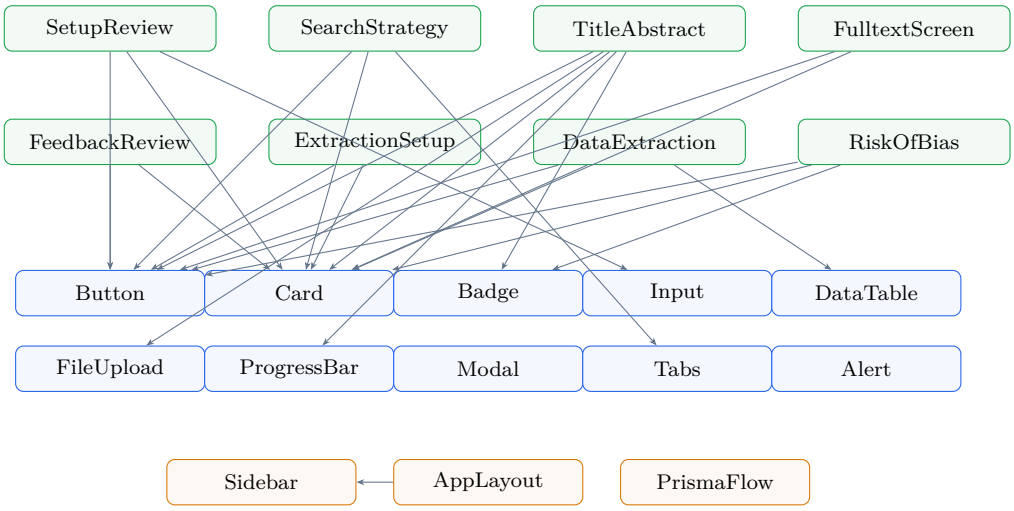


Figure 6: Frontend component dependency graph (simplified).

# 14 Summary Statistics

| Metric | Count |
|---|---|
| **Original Core (Python)** | |
| Core modules | 9 |
| Python source files | 40+ |
| Pydantic models | 30+ |
| LLM prompt templates | 16 |
| RoB tool templates | 5 |
| SQLite tables | 13 |
| **FastAPI Backend** | |
| API routers | 7 |
| REST endpoints | 30 |
| Pydantic request/response models | 35+ |
| Background task workers | 3 |
| **React Frontend** | |
| Page components | 9 |
| Reusable UI components | 11 |
| Layout components | 2 |
| Special components | 1 (PrismaFlow) |
| TypeScript interfaces | 15 |
| React Query hooks | 30+ |
| API service methods | 30+ |
| Routes | 10 |
| **Build Output** | |
| CSS bundle | 38.8 KB |
| JS bundle | 386 KB (116 KB gzipped) |

Table 29: Project statistics.